

Let's look at a basic example of `map()` that squares all of the numbers in an RDD (Examples 3-26 through 3-28).

Example 3-26. Python squaring the values in an RDD

```
nums = sc.parallelize([1, 2, 3, 4])
squared = nums.map(lambda x: x * x).collect()
for num in squared:
    print "%i " % (num)
```

Example 3-27. Scala squaring the values in an RDD

```
val input = sc.parallelize(List(1, 2, 3, 4))
val result = input.map(x => x * x)
println(result.collect().mkString(", "))
```

Example 3-28. Java squaring the values in an RDD

```
JavaRDD<Integer> rdd = sc.parallelize(Arrays.asList(1, 2, 3, 4));
JavaRDD<Integer> result = rdd.map(new Function<Integer, Integer>() {
    public Integer call(Integer x) { return x*x; }
});
System.out.println(StringUtils.join(result.collect(), ", "));
```

Sometimes we want to produce multiple output elements for each input element. The operation to do this is called `flatMap()`. As with `map()`, the function we provide to `flatMap()` is called individually for each element in our input RDD. Instead of returning a single element, we return an iterator with our return values. Rather than producing an RDD of iterators, we get back an RDD that consists of the elements from all of the iterators. A simple usage of `flatMap()` is splitting up an input string into words, as shown in Examples 3-29 through 3-31.

Example 3-29. flatMap() in Python, splitting lines into words

```
lines = sc.parallelize(["hello world", "hi"])
words = lines.flatMap(lambda line: line.split(" "))
words.first() # returns "hello"
```

Example 3-30. flatMap() in Scala, splitting lines into multiple words

```
val lines = sc.parallelize(List("hello world", "hi"))
val words = lines.flatMap(line => line.split(" "))
words.first() // returns "hello"
```

Example 3-31. `flatMap()` in Java, splitting lines into multiple words

```
JavaRDD<String> lines = sc.parallelize(Arrays.asList("hello world", "hi"));
JavaRDD<String> words = lines.flatMap(new FlatMapFunction<String, String>() {
    public Iterable<String> call(String line) {
        return Arrays.asList(line.split(" "));
    }
});
words.first(); // returns "hello"
```

We illustrate the difference between `flatMap()` and `map()` in Figure 3-3. You can think of `flatMap()` as “flattening” the iterators returned to it, so that instead of ending up with an RDD of lists we have an RDD of the elements in those lists.

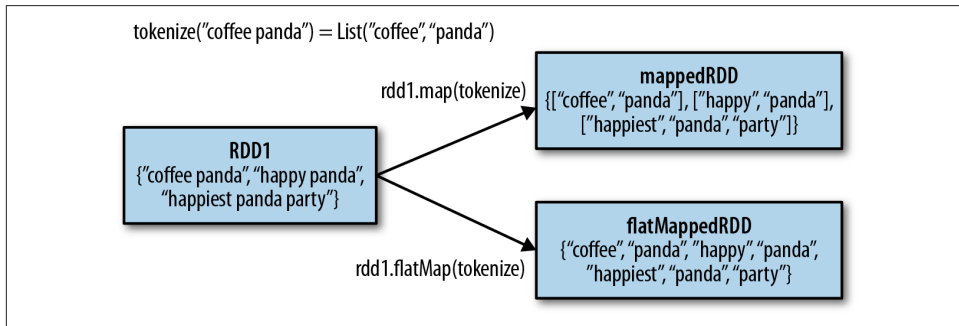


Figure 3-3. Difference between `flatMap()` and `map()` on an RDD

Pseudo set operations

RDDs support many of the operations of mathematical sets, such as union and intersection, even when the RDDs themselves are not properly sets. Four operations are shown in Figure 3-4. It’s important to note that all of these operations require that the RDDs being operated on are of the same type.

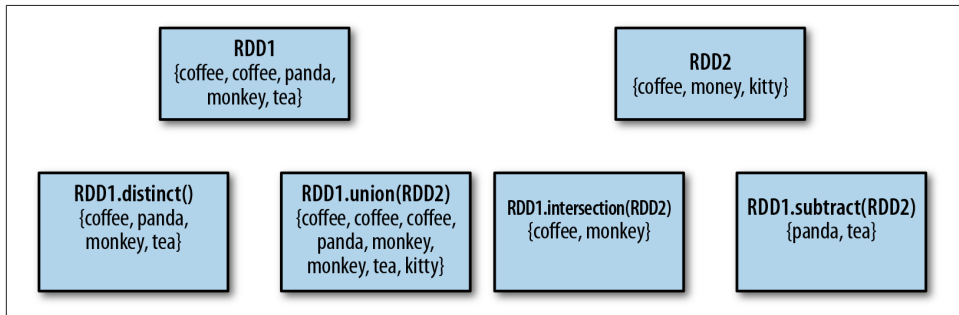


Figure 3-4. Some simple set operations

The set property most frequently missing from our RDDs is the uniqueness of elements, as we often have duplicates. If we want only unique elements we can use the `RDD.distinct()` transformation to produce a new RDD with only distinct items. Note that `distinct()` is expensive, however, as it requires shuffling all the data over the network to ensure that we receive only one copy of each element. Shuffling, and how to avoid it, is discussed in more detail in [Chapter 4](#).

The simplest set operation is `union(other)`, which gives back an RDD consisting of the data from both sources. This can be useful in a number of use cases, such as processing logfiles from many sources. Unlike the mathematical `union()`, if there are duplicates in the input RDDs, the result of Spark's `union()` will contain duplicates (which we can fix if desired with `distinct()`).

Spark also provides an `intersection(other)` method, which returns only elements in both RDDs. `intersection()` also removes all duplicates (including duplicates from a single RDD) while running. While `intersection()` and `union()` are two similar concepts, the performance of `intersection()` is much worse since it requires a shuffle over the network to identify common elements.

Sometimes we need to remove some data from consideration. The `subtract(other)` function takes in another RDD and returns an RDD that has only values present in the first RDD and not the second RDD. Like `intersection()`, it performs a shuffle.

We can also compute a Cartesian product between two RDDs, as shown in [Figure 3-5](#). The `cartesian(other)` transformation returns all possible pairs of (a, b) where a is in the source RDD and b is in the other RDD. The Cartesian product can be useful when we wish to consider the similarity between all possible pairs, such as computing every user's expected interest in each offer. We can also take the Cartesian product of an RDD with itself, which can be useful for tasks like user similarity. Be warned, however, that the Cartesian product is *very expensive* for large RDDs.

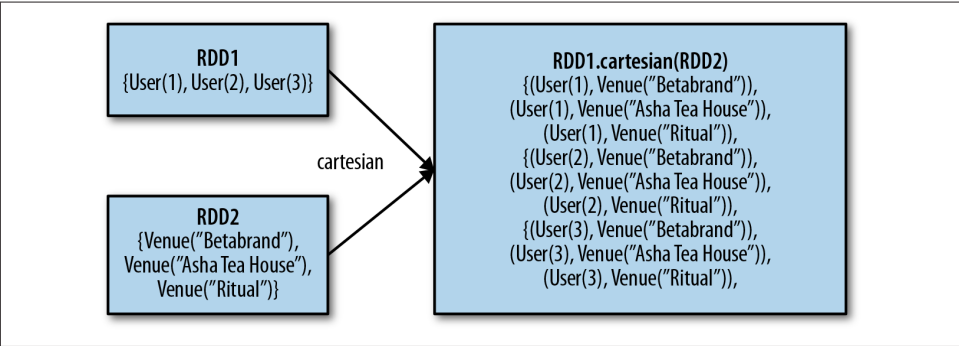


Figure 3-5. Cartesian product between two RDDs

Tables [3-2](#) and [3-3](#) summarize these and other common RDD transformations.

Table 3-2. Basic RDD transformations on an RDD containing {1, 2, 3, 3}

Function name	Purpose	Example	Result
map()	Apply a function to each element in the RDD and return an RDD of the result.	<code>rdd.map(x => x + 1)</code>	{2, 3, 4, 4}
flatMap()	Apply a function to each element in the RDD and return an RDD of the contents of the iterators returned. Often used to extract words.	<code>rdd.flatMap(x => x.to(3))</code>	{1, 2, 3, 2, 3, 3, 3}
filter()	Return an RDD consisting of only elements that pass the condition passed to filter().	<code>rdd.filter(x => x != 1)</code>	{2, 3, 3}
distinct()	Remove duplicates.	<code>rdd.distinct()</code>	{1, 2, 3}
sample(withReplacement, fraction, [seed])	Sample an RDD, with or without replacement.	<code>rdd.sample(false, 0.5)</code>	Nondeterministic

Table 3-3. Two-RDD transformations on RDDs containing {1, 2, 3} and {3, 4, 5}

Function name	Purpose	Example	Result
union()	Produce an RDD containing elements from both RDDs.	<code>rdd.union(other)</code>	{1, 2, 3, 3, 4, 5}
intersection()	RDD containing only elements found in both RDDs.	<code>rdd.intersection(other)</code>	{3}
subtract()	Remove the contents of one RDD (e.g., remove training data).	<code>rdd.subtract(other)</code>	{1, 2}
cartesian()	Cartesian product with the other RDD.	<code>rdd.cartesian(other)</code>	{(1, 3), (1, 4), ... (3,5)}

Actions

The most common action on basic RDDs you will likely use is `reduce()`, which takes a function that operates on two elements of the type in your RDD and returns a new element of the same type. A simple example of such a function is `+`, which we can use to sum our RDD. With `reduce()`, we can easily sum the elements of our RDD, count the number of elements, and perform other types of aggregations (see Examples 3-32 through 3-34).

Example 3-32. `reduce()` in Python

```
sum = rdd.reduce(lambda x, y: x + y)
```

Example 3-33. `reduce()` in Scala

```
val sum = rdd.reduce((x, y) => x + y)
```

Example 3-34. `reduce()` in Java

```
Integer sum = rdd.reduce(new Function2<Integer, Integer, Integer>() {  
    public Integer call(Integer x, Integer y) { return x + y; }  
});
```

Similar to `reduce()` is `fold()`, which also takes a function with the same signature as needed for `reduce()`, but in addition takes a “zero value” to be used for the initial call on each partition. The zero value you provide should be the identity element for your operation; that is, applying it multiple times with your function should not change the value (e.g., 0 for +, 1 for *, or an empty list for concatenation).



You can minimize object creation in `fold()` by modifying and returning the first of the two parameters in place. However, you should not modify the second parameter.

Both `fold()` and `reduce()` require that the return type of our result be the same type as that of the elements in the RDD we are operating over. This works well for operations like `sum`, but sometimes we want to return a different type. For example, when computing a running average, we need to keep track of both the count so far and the number of elements, which requires us to return a pair. We could work around this by first using `map()` where we transform every element into the element and the number 1, which is the type we want to return, so that the `reduce()` function can work on pairs.

The `aggregate()` function frees us from the constraint of having the return be the same type as the RDD we are working on. With `aggregate()`, like `fold()`, we supply an initial zero value of the type we want to return. We then supply a function to combine the elements from our RDD with the accumulator. Finally, we need to supply a second function to merge two accumulators, given that each node accumulates its own results locally.

We can use `aggregate()` to compute the average of an RDD, avoiding a `map()` before the `fold()`, as shown in Examples 3-35 through 3-37.

Example 3-35. aggregate() in Python

```
sumCount = nums.aggregate((0, 0),
    (lambda acc, value: (acc[0] + value, acc[1] + 1),
    (lambda acc1, acc2: (acc1[0] + acc2[0], acc1[1] + acc2[1]))))
return sumCount[0] / float(sumCount[1])
```

Example 3-36. aggregate() in Scala

```
val result = input.aggregate((0, 0))((
    (acc, value) => (acc._1 + value, acc._2 + 1),
    (acc1, acc2) => (acc1._1 + acc2._1, acc1._2 + acc2._2))
val avg = result._1 / result._2.toDouble
```

Example 3-37. aggregate() in Java

```
class AvgCount implements Serializable {
    public AvgCount(int total, int num) {
        this.total = total;
        this.num = num;
    }
    public int total;
    public int num;
    public double avg() {
        return total / (double) num;
    }
}

Function2<AvgCount, Integer, AvgCount> addAndCount =
    new Function2<AvgCount, Integer, AvgCount>() {
        public AvgCount call(AvgCount a, Integer x) {
            a.total += x;
            a.num += 1;
            return a;
        }
    };

Function2<AvgCount, AvgCount, AvgCount> combine =
    new Function2<AvgCount, AvgCount, AvgCount>() {
        public AvgCount call(AvgCount a, AvgCount b) {
            a.total += b.total;
            a.num += b.num;
            return a;
        }
    };

AvgCount initial = new AvgCount(0, 0);
AvgCount result = rdd.aggregate(initial, addAndCount, combine);
System.out.println(result.avg());
```

Some actions on RDDs return some or all of the data to our driver program in the form of a regular collection or value.

The simplest and most common operation that returns data to our driver program is `collect()`, which returns the entire RDD's contents. `collect()` is commonly used in unit tests where the entire contents of the RDD are expected to fit in memory, as that makes it easy to compare the value of our RDD with our expected result. `collect()` suffers from the restriction that all of your data must fit on a single machine, as it all needs to be copied to the driver.

`take(n)` returns n elements from the RDD and attempts to minimize the number of partitions it accesses, so it may represent a biased collection. It's important to note that these operations do not return the elements in the order you might expect.

These operations are useful for unit tests and quick debugging, but may introduce bottlenecks when you're dealing with large amounts of data.

If there is an ordering defined on our data, we can also extract the top elements from an RDD using `top()`. `top()` will use the default ordering on the data, but we can supply our own comparison function to extract the top elements.

Sometimes we need a sample of our data in our driver program. The `takeSample(withReplacement, num, seed)` function allows us to take a sample of our data either with or without replacement.

Sometimes it is useful to perform an action on all of the elements in the RDD, but without returning any result to the driver program. A good example of this would be posting JSON to a webserver or inserting records into a database. In either case, the `foreach()` action lets us perform computations on each element in the RDD without bringing it back locally.

The further standard operations on a basic RDD all behave pretty much exactly as you would imagine from their name. `count()` returns a count of the elements, and `countByValue()` returns a map of each unique value to its count. [Table 3-4](#) summarizes these and other actions.

Table 3-4. Basic actions on an RDD containing {1, 2, 3, 3}

Function name	Purpose	Example	Result
<code>collect()</code>	Return all elements from the RDD.	<code>rdd.collect()</code>	{1, 2, 3, 3}
<code>count()</code>	Number of elements in the RDD.	<code>rdd.count()</code>	4
<code>countByValue()</code>	Number of times each element occurs in the RDD.	<code>rdd.countByValue()</code>	{(1, 1), (2, 1), (3, 2)}

Function name	Purpose	Example	Result
<code>take(num)</code>	Return num elements from the RDD.	<code>rdd.take(2)</code>	{1, 2}
<code>top(num)</code>	Return the top num elements the RDD.	<code>rdd.top(2)</code>	{3, 3}
<code>takeOrdered(num)(ordering)</code>	Return num elements based on provided ordering.	<code>rdd.takeOrdered(2)(myOrdering)</code>	{3, 3}
<code>takeSample(withReplacement, num, [seed])</code>	Return num elements at random.	<code>rdd.takeSample(false, 1)</code>	Nondeterministic
<code>reduce(func)</code>	Combine the elements of the RDD together in parallel (e.g., sum).	<code>rdd.reduce((x, y) => x + y)</code>	9
<code>fold(zero)(func)</code>	Same as <code>reduce()</code> but with the provided zero value.	<code>rdd.fold(0)((x, y) => x + y)</code>	9
<code>aggregate(zeroValue)(seqOp, combOp)</code>	Similar to <code>reduce()</code> but used to return a different type.	<code>rdd.aggregate((0, 0))((x, y) => (x._1 + y, x._2 + 1), (x, y) => (x._1 + y._1, x._2 + y._2))</code>	(9, 4)
<code>foreach(func)</code>	Apply the provided function to each element of the RDD.	<code>rdd.foreach(func)</code>	Nothing

Converting Between RDD Types

Some functions are available only on certain types of RDDs, such as `mean()` and `variance()` on numeric RDDs or `join()` on key/value pair RDDs. We will cover these special functions for **numeric data** in [Chapter 6](#) and pair RDDs in [Chapter 4](#). In Scala and Java, these methods aren't defined on the standard RDD class, so to access this additional functionality we have to make sure we get the correct specialized class.

Scala

In Scala the conversion to RDDs with special functions (e.g., to expose numeric functions on an `RDD[Double]`) is handled automatically using implicit conversions. As mentioned in “[Initializing a SparkContext](#)” on page 17, we need to add `import org.apache.spark.SparkContext._` for these conversions to work. You can see the implicit conversions listed in the [SparkContext object’s ScalaDoc](#). These implicits turn an RDD into various wrapper classes, such as `DoubleRDDFunctions` (for RDDs of numeric data) and `PairRDDFunctions` (for key/value pairs), to expose additional functions such as `mean()` and `variance()`.

Implicits, while quite powerful, can sometimes be confusing. If you call a function like `mean()` on an RDD, you might look at the [Scaladocs for the RDD class](#) and notice there is no `mean()` function. The call manages to succeed because of implicit conversions between `RDD[Double]` and `DoubleRDDFunctions`. When searching for functions on your RDD in Scaladoc, make sure to look at functions that are available in these wrapper classes.

Java

In Java the conversion between the specialized types of RDDs is a bit more explicit. In particular, there are special classes called `JavaDoubleRDD` and `JavaPairRDD` for RDDs of these types, with extra methods for these types of data. This has the benefit of giving you a greater understanding of what exactly is going on, but can be a bit more cumbersome.

To construct RDDs of these special types, instead of always using the `Function` class we will need to use specialized versions. If we want to create a `DoubleRDD` from an RDD of type `T`, rather than using `Function<T, Double>` we use `DoubleFunction<T>`. [Table 3-5](#) shows the specialized functions and their uses.

We also need to call different functions on our RDD (so we can’t just create a `DoubleFunction` and pass it to `map()`). When we want a `DoubleRDD` back, instead of calling `map()`, we need to call `mapToDouble()` with the same pattern all of the other functions follow.

Table 3-5. Java interfaces for type-specific functions

Function name	Equivalent function* <code><A, B,...></code>	Usage
<code>DoubleFlatMapFunction<T></code>	<code>Function<T, Iterable<Double>></code>	<code>DoubleRDD</code> from a <code>flatMapToDouble</code>
<code>DoubleFunction<T></code>	<code>Function<T, double></code>	<code>DoubleRDD</code> from <code>mapToDouble</code>

Function name	Equivalent function*<A, B,...>	Usage
PairFlatMapFunction<T, K, V>	Function<T, Iterable<Tuple2<K, V>>>	PairRDD<K, V> from a flatMapToPair
PairFunction<T, K, V>	Function<T, Tuple2<K, V>>	PairRDD<K, V> from a mapToPair

We can modify [Example 3-28](#), where we squared an RDD of numbers, to produce a `JavaDoubleRDD`, as shown in [Example 3-38](#). This gives us access to the additional `DoubleRDD` specific functions like `mean()` and `variance()`.

Example 3-38. Creating DoubleRDD in Java

```
JavaDoubleRDD result = rdd.mapToDouble(
    new DoubleFunction<Integer>() {
        public double call(Integer x) {
            return (double) x * x;
        }
    });
System.out.println(result.mean());
```

Python

The Python API is structured differently than Java and Scala. In Python all of the functions are implemented on the base RDD class but will fail at runtime if the type of data in the RDD is incorrect.

Persistence (Caching)

As discussed earlier, Spark RDDs are lazily evaluated, and sometimes we may wish to use the same RDD multiple times. If we do this naively, Spark will recompute the RDD and all of its dependencies each time we call an action on the RDD. This can be especially expensive for iterative algorithms, which look at the data many times. Another trivial example would be doing a count and then writing out the same RDD, as shown in [Example 3-39](#).

Example 3-39. Double execution in Scala

```
val result = input.map(x => x*x)
println(result.count())
println(result.collect().mkString(", "))
```

To avoid computing an RDD multiple times, we can ask Spark to *persist* the data. When we ask Spark to persist an RDD, the nodes that compute the RDD store their

partitions. If a node that has data persisted on it fails, Spark will recompute the lost partitions of the data when needed. We can also replicate our data on multiple nodes if we want to be able to handle node failure without slowdown.

Spark has many levels of persistence to choose from based on what our goals are, as you can see in [Table 3-6](#). In Scala ([Example 3-40](#)) and Java, the default `persist()` will store the data in the JVM heap as unserialized objects. In Python, we always serialize the data that persist stores, so the default is instead stored in the JVM heap as pickled objects. When we write data out to disk or off-heap storage, that data is also always serialized.

Table 3-6. Persistence levels from `org.apache.spark.storage.StorageLevel` and `pyspark.StorageLevel`; if desired we can replicate the data on two machines by adding `_2` to the end of the storage level

Level	Space used	CPU time	In memory	On disk	Comments
MEMORY_ONLY	High	Low	Y	N	
MEMORY_ONLY_SER	Low	High	Y	N	
MEMORY_AND_DISK	High	Medium	Some	Some	Spills to disk if there is too much data to fit in memory.
MEMORY_AND_DISK_SER	Low	High	Some	Some	Spills to disk if there is too much data to fit in memory. Stores serialized representation in memory.
DISK_ONLY	Low	High	N	Y	



Off-heap caching is experimental and uses [Tachyon](#). If you are interested in off-heap caching with Spark, [take a look at the Running Spark on Tachyon guide](#).

Example 3-40. `persist()` in Scala

```
val result = input.map(x => x * x)
result.persist(StorageLevel.DISK_ONLY)
println(result.count())
println(result.collect().mkString(", "))
```

Notice that we called `persist()` on the RDD before the first action. The `persist()` call on its own doesn't force evaluation.

If you attempt to cache too much data to fit in memory, Spark will automatically evict old partitions using a Least Recently Used (LRU) cache policy. For the memory-only storage levels, it will recompute these partitions the next time they are accessed, while for the memory-and-disk ones, it will write them out to disk. In either case, this means that you don't have to worry about your job breaking if you ask Spark to cache too much data. However, caching unnecessary data can lead to eviction of useful data and more recomputation time.

Finally, RDDs come with a method called `unpersist()` that lets you manually remove them from the cache.

Conclusion

In this chapter, we have covered the RDD execution model and a large number of common operations on RDDs. If you have gotten here, congratulations—you've learned all the core concepts of working in Spark. In the next chapter, we'll cover a special set of operations available on RDDs of key/value pairs, which are the most common way to aggregate or group together data in parallel. After that, we discuss input and output from a variety of data sources, and more advanced topics in working with `SparkContext`.